

Question 1 : What is the output ?

```
func main() {
    ch := make(chan int, 3)
    ch <- 1
    ch <- 2

    fmt.Println(len(ch), cap(ch))

    <-ch
    fmt.Println(len(ch), cap(ch))

    ch <- 3
    ch <- 4
    fmt.Println(len(ch), cap(ch))
}
```

```
func main() {
    ch := make(chan int, 3)
    ch <- 10
    ch <- 20
    ch <- 30

    for v := range ch {
        fmt.Println(v)
    }
    fmt.Println("after loop")
}
```

```
func main() {
    var a chan int
    defer func() { fmt.Println("recovered:", recover()) }()
    close(a)
}
```

Question 2 : Write Go Lang Code To Solve These Problems:

1-Given a slice `nums []int` and a window size `k`, return a slice of the maximum value in each window of size `k` as it slides from left to right.

**Input:** `nums = [1,3,-1,-3,5,3,6,7], k = 3` → **Output:**  
`[3,3,5,5,6,7]`

2- Given a list of words, group the anagrams together.

**Input:** `["eat", "tea", "tan", "ate", "nat", "bat"]`

**Output:** `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]`

3- Define a `Student` struct (name, id, grades map). Implement:

- A method `Average() float64`
- A function `TopStudent(students []Student) *Student` returning a pointer to the student with the highest average.

4-Implement `sort.Interface` for a `ByAge []Person` type to sort people by age. Then implement a custom `Ranker` interface to sort any rankable collection.

5- **Retry with Exponential Backoff** .

Write `Retry(times int, fn func() error) error` that calls `fn` up to `times` times, returning the last error if all attempts fail. Wrap the final error with `fmt.Errorf("after %d attempts: %w", ...)`.

6- If a goroutine panics and nothing recovers it, what happens to the program?

The whole program crashes — `recover` only works in the goroutine where the panic occurred. Write a worker pool where each worker recovers its own panics and reports them on an error channel.

7- Implement a rate limiter that allows at most `rate` operations per second using `time.Ticker`, and process 20 tasks through it.

8- Split a large slice into N chunks, sum each in its own goroutine using `sync.WaitGroup`, and combine the results.

```
func ParallelSum(nums []int, workers int) int
```

### Question 3 : Mini Project

- `Task` struct with `ID`, `Name`, `Payload map[string]any`, and `Priority int`.
- A `Handler interface` (`Handle(ctx context.Context, t Task) error`) with at least 3 implementations (e.g. `EmailHandler`, `ResizeImageHandler`, `ReportHandler`), registered in a `map` keyed by task type.
- A **worker pool** consuming tasks from a **channel**, with a configurable number of workers and graceful shutdown using `context`.
- Handlers sometimes return errors; use **custom error types** and **wrapping**. Failures with `RetryableError` are retried up to 3 times.
- Handlers sometimes **panic**; recover them per task so one bad task never kills a worker, and convert them to errors.
- Collect results into a **slice** and print a summary: succeeded, failed, retried, and panicked counts, plus average duration.
- Protect shared stats with a mutex or atomics.

Good Luck 😊